3. Modernization(blueprint)

This document is about transforming the existing infrastructure into a **scalable, hybrid and reliable infra** that supports high-velocity deployments, real-time workloads, and long-term operational efficiency.

> “The modernization phase transitions the organization from infrastructure management to platform engineering, enabling scalable, resilient, and telecom-grade system design aligned with real-time workload demands.”
>
> This phase focuses on:
>
> - Standardizing infrastructure
> - Decoupling system components
> - Building for **scale, resilience, and observability**.

With all this, the expect outcome to achieve

1. Cost: Sustained cost efficiency via hybrid placement
2. Performance:
   - Low-latency media handling
   - Optimized network paths
3. Security
   - Continuous visibility
   - Progressive enforcement
4. Engineering Velocity
   - Faster onboarding
   - Standardized platform driven deployments
   - Reduced manual operations.

Current State: Fast deployment pipeline (retain)
Modern State:

- Integrated with:
  - GitOps
  - Feedback pipelines
  - Blue-Green Deployment
  - Automated rollbacks
- **Future State**: IDPs, Platform Engineering Flow

Note: Reality Check (Important)
Donts:

1. Workload-Aware Architecture

- *“move everything to Kubernetes”*, Use Kubernetes **where it adds value**, not everywhere.

```
Scope of Kubernetes:
Suitable Workloads
- Signaling services
- APIs
- Control plane components
- Stateless services

Media workloads:
- Use **hostNetwork**
- Deploy via **DaemonSets**
- Avoid overlay networking
```

- *“rewrite everything”*
  Dos:
- *Segregate and deploy wherever necessary* , Place workloads based on:
  - latency
  - bandwidth
  - cost
- Move from: - “managing servers” → “operating platforms” without interruption:
  - *retain what's already working (high velocity deployment) without disturbing and disrupting*
  - *reduce toils, strip and restore the necessary.*

Achieving:

- **Resource Rationalization**
- **Hybrid Architecture Enablement**
- **Platform Standardization (Kubernetes + GitOps)**
- **DevSecOps Integration (Non-blocking)**
- **Observability & Governance**

Modernization Action Items:

1. **Resource Segregation**
2. **Internal Migration after segregation (instance - migrate to graviton)**
3. **Lift and Shift Migration**
4. **On-prem Setup (kubeadm) - Nodes spec to discuss (HA or Multi-Node Model)**
5. **Site-to-site VPN Tunnel or Self managed VPN (Wireguard/Tailscale) between cloud and on-prem**
6. **Enforce Local Dev Environment Setup - SonarLint, Lens, Teleport**
7. **Hub Cluster Setup - K8s/Docker - OnPrem**
8. **IaC - Terraform**
9. **Multi-Cluster Setup (GitOps using ArgoCD)**
10. **Services Deployment**
    - **Dedicated Git Server - Gitea,**
    - **Object Storage Server - MinIO/CubeFS,**
    - **Block storage server - Longhorn,**
    - **API Gateway - Traefik**
    - **IAM - Keycloak**
    - **PM - Plane/Mattermost/Open Project**
    - **SecOps**
      - **SAST - SonarQube,**
      - **SCA - Trivy, Docker Scout**
      - **DAST - OWASP ZAP,**
    - **Container registry - Harbor**
    - **DBMS - Phpmyadmin/Adminer**
11. **SRE Observability Stack - Prometheus, Grafana/ELK/Datadog**
12. **Cluster Visualization and management (Headlamp and Teleport)**

1. Resource Segregation

Organizing all the workloads into clearly defined categories and environments. cuz, Current setup shows **high fragmentation (29 regions, 35 VPCs, scattered resources)** leading to:

- Cost leakage,
- Operational confusion
- Lack of ownership

  After organizing, it helps in,
  - Clear workload placement (cloud vs on-prem)
  - Easier cost tracking and governance

- Reduces unused/stale resources

- [ ] Lockdown AWS regions (target: 2–3 active regions)
- [ ] Export full resource inventory (tag-based / CLI)
- [ ] Identify and categorize workloads:
  - [ ] Telco (RTP, TURN, SIP)
  - [ ] Application (APIs, services, three-tier)
  - [ ] Data (DB, storage)
  - [ ] Managed Services.
- [ ] Map workloads → placement(environments):
  - [ ] Cloud (AWS)
  - [ ] On-Prem
- [ ] Segregate the diverse workloads and cherry-pick which has to stay on cloud, which has to be migrated.
- [ ] Tag all resources:
  - [ ] Owner
  - [ ] Environment
  - [ ] Workload type
- [ ] Review
- [ ] Define cluster boundaries based on workloads:

Cluster	Purpose
AWS Cluster	APIs, Signaling, Application workloads
On-Prem Cluster	Media, SIP workloads
Hub Services Cluster	Self hosted tools, managed services and more

---

2. Internal Migration after segregation (instance - migrate to graviton)

Migrating compatible workloads from x86 to **ARM-based AWS instances(graviton)**. Necessary because, Compute is a major cost driver → ARM based CPU offers **~20–40% cost savings**.

- [ ] Extract SBOM for all workloads
- [ ] Assess and evaluate workloads for compatibility (AWS Graviton Instances) using migration accelerator graviton
- [ ] Identify compatible workloads

- [ ] Benchmark them
- [ ] Segregate them
- [ ] Perform migration in batch: Run batch processes for automating the migration by switching instances
- [ ] Test and Evaluate the workloads, Validate:
  - [ ] Functional testing
  - [ ] Performance testing
- [ ] Monitor:
  - [ ] Cost reduction
  - [ ] CPU utilization
- [ ] Calculate the cost difference

---

3. **Lift and Shift Migration** (cloud to bare-metal)

Moving existing workloads into structured environments **without disturbing them**.
it helps with
- Faster migration
- Low risk
- Enables future containerization / scaling
because,

- Current workloads are unmanaged / inconsistent
- Need quick standardization before deeper modernization

- [ ] Identify legacy workloads (VM-based / unmanaged / Stateful/ Immutable)
- [ ] Classify:
  - [ ] Containerizable
  - [ ] Non-containerizable
- [ ] Prepare base images
- [ ] Containerize selected services
- [ ] Deploy to staging environment
- [ ] Validate behaviour:
  - [ ] Service functionality
  - [ ] Network behavior
- [ ] Migrate to targets:
  - [ ] AWS cluster OR On-prem cluster (based on type)

---

4. On-prem Setup (`kubeadm`) - Nodes spec to discuss (HA or Multi-Node Model)

Setting up a Kubernetes cluster in on-prem datacenter using kubeadm.

- Reduces bandwidth cost
- Full infra control
- Low latency, High bandwidth for domestic distribution
  Because, cloud is expensive for media traffic,

To setup 2 Clusters on-prem:

- [ ] Hub Cluster - Self hosted, Managed Services
- [ ] Spoke Cluster - Production Cluster
- [ ] Finalize node specs:
  - [ ] Minimum: 1 master + 2 workers
  - [ ] Future: HA control plane
- [ ] Provision infrastructure (bare metal / VMs)
- [ ] Configure each node - Kubeadm Setup
- [ ] Initialize cluster using kubeadm with Calico CNI,
- [ ] Configure:
  - [ ] `hostNetwork` support (for RTP)
- [ ] configure `.kubeconfig` for context (multi-cluster compatibility)
- [ ] add clusters to ArgoCD.
- [ ] Validate:
  - [ ] Node readiness
  - [ ] Pod scheduling

---

5. Site-to-site VPN Tunnel or Self managed VPN (Wireguard/Tailscale) between cloud and on-prem - Hybrid Infra setup

Secure network connection between AWS and on-prem environments.
It establishes seamless communication between the Cloud and DC. This helps in

- Enables hybrid architecture
- Secure communication
- Low-latency routing

- [ ] Choose after evaluating a VPN model:
  - [ ] WireGuard (recommended, lesser net hopping layers harder to setup and manage, free)
  - [ ] Tailscale (optional, easy to setup and manage, proprietary)
- [ ] Setup VPN server (on-prem)
- [ ] Setup and Configure AWS peering connection (Site-to-site VPN Tunnel)
- [ ] Establish secure tunnel
- [ ] Validate:
  - [ ] Pod-to-pod communication
  - [ ] Service reachability
- [ ] Optimize:
  - [ ] Latency
  - [ ] Routing rules

6.Enforce Local Dev Environment Standardization

Defining a consistent developer setup across the team. Because, the current workloads sits under - Inconsistent environments - prone to bugs, drift, No prior QC baseline (only limited to JMeter or JUnit).

Having a defined setup helps,

- Improves code quality early
- Reduces deployment issues
- Standardizes developer workflow
- helps to go Agile
- helps to comply with shift-left security

- [ ] Define dev environment baseline
- [ ] Enforce tools:
  - [ ] Wireguard/Tailscale
    - [ ] Termix/WinSCP
    - [ ] SonarLint (code quality)
    - [ ] Lens (K8s visibility)
    - [ ] Teleport (configure secure access for teams)
- [ ] Provide setup documentation
- [ ] Ensure:
  - [ ] Consistent local testing

- [ ] Cluster access standardization

---

7. Hub Cluster Setup - K8s/Docker - OnPrem

A central Kubernetes cluster hosting internal tools and platform services. Having a unified and dedicated cluster for running services which does not bothers prod workloads. On its own paces.

Goal:

- Single place for:
  - Self hosted developer tools
  - Internal managed services
- Easier management
- Resource isolation
- Easier distribution

- [ ] Design hub cluster role:
  - [ ] Internal tools
  - [ ] Self Hosted
  - [ ] CI/CD services
  - [ ] Knowledge Hub
- [ ] Deploy base Kubernetes cluster
- [ ] Configure storage:
  - [ ] Longhorn
  - [ ] CubeFS
- [ ] Setup networking:
  - [ ] Internal ingress
  - [ ] Traefik
- [ ] Validate cluster stability

---

8. IaC - Terraform

Managing infrastructure using code instead of manual setup. Primarily for Reproducibility. It helps with,

- Consistent infra creation
- Version control

- Eliminates configuration drift

> **because, Current infra is manual → error-prone.**

- [ ] Define Terraform structure:
  - [ ] Modules (network, compute, storage)
- [ ] Import existing AWS resources (if needed)
- [ ] Configure Local `AWS-CLI`, `eksctl` and more for hybrid infra management,
- [ ] Create reusable modules:
  - [ ] VPC
  - [ ] EC2
  - [ ] EKS (if applicable)
  - [ ] S3
- [ ] Setup remote backend (S3 + locking)
- [ ] Validate:
  - [ ] Plan / Apply consistency
- [ ] Reproduce the existing infra with templates,
- [ ] Enforce:
  - [ ] IaC
    - [ ] No manual provisioning

---

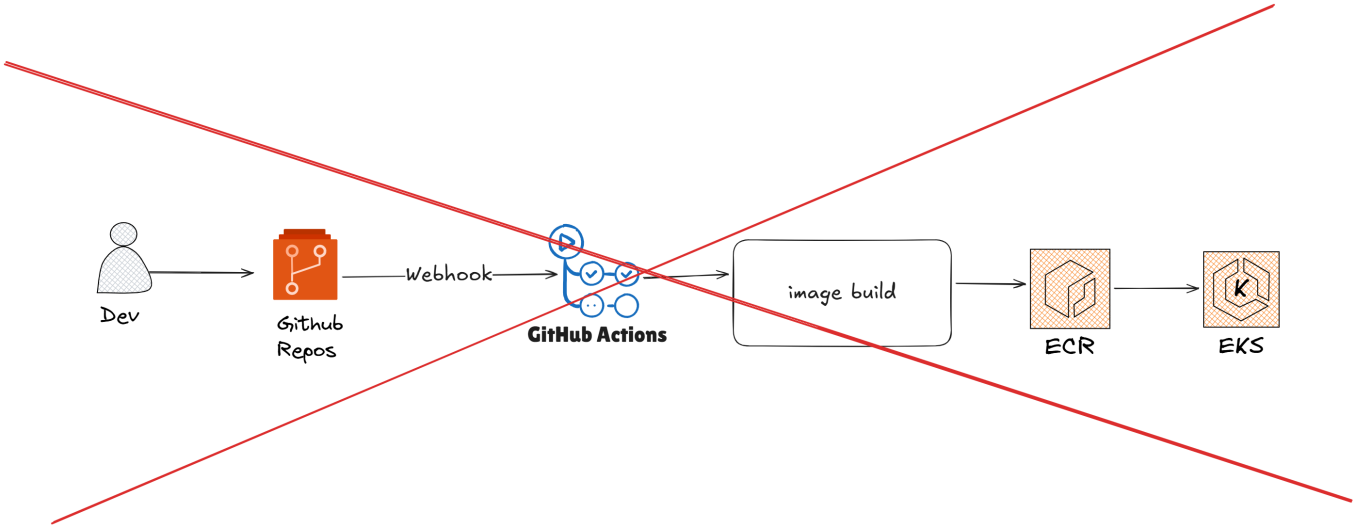
9. Multi-Cluster Setup (GitOps using ArgoCD)

GitOps-Driven Multi-Cluster Management:

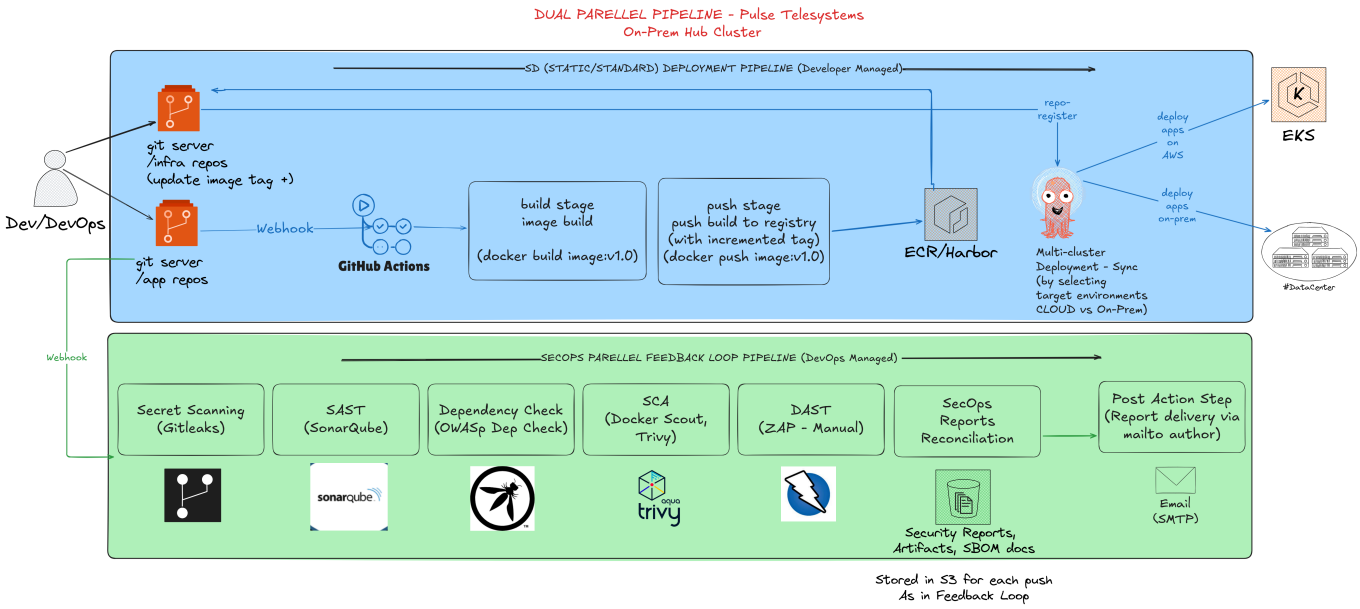
Managing deployments across multiple clusters using Git as the source of truth. For - Hybrid setup → multiple clusters and Manual deployment = inconsistent.

Architectural Deployment Diagram:

Replacing:



with:



> **This might seem very specific and maybe unconventional than traditional CI/CD Pipelines. This is planned because,**
> 1. We get constant, frequent feedback loop support tickets and feature request where we fix the same right away with minimal to no checking hops. ==
> 2. ==Since it has been consistent and work well for us for our specific usecase, no matter what QA/QC is unavoidable and highly necessary for any means. For such case, came up with ==
> 3. ==Feedback Loop Pipeline - where it goes through all the checks and provides with all the security reports, CVE lists, Code Coverage and more in detail for each deployment in a unified visibility.
> 4. Developers can later fix the issue and sort the same in ease which again goes through Feedback Loop for absolute refinement.

Benefits:

- Consistent deployments across environments
- Version-controlled infrastructure
- Easy rollback and auditability

- [ ] Install ArgoCD in hub cluster
- [ ] Register clusters:
  - [ ] AWS cluster
  - [ ] On-prem cluster
- [ ] Create Git repository structure:
  - [ ] apps/
  - [ ] infra/
- [ ] Define application manifests (Helm/Kustomize)
- [ ] Deploy via ArgoCD
- [ ] Validate:
  - [ ] Sync status
  - [ ] Rollback capability

---

10. **Platform Services Deployment - Hub Cluster/Docker**

Deploying essential internal services required to run the platform. Is to hnave full control over services, Cost optimization, unified ecosystem.

necessary to:
- Avoid reliance on external billable tools
- Selfhosting and Autonomy
- Build IDPs and Platform tools

1. [ ] **Dedicated Git Server - Gitea,**
2. [ ] CI/CD - Jenkins, Github Actions
3. [ ] Management - Portainer
4. [ ] **Object Storage Server - MinIO/CubeFS,**
5. [ ] **Block storage server - Longhorn,**
6. [ ] **API Gateway - Traefik**
7. [ ] Configure routing rules
8. [ ] Setup TLS (certificate-manager)
9. [ ] **IAM - Keycloak**
   Configure:
   - [ ] Users

- [ ] Roles
- [ ] SSO (optional)

10. [ ] **PM/Collaboration - Plane/Mattermost/Open Project**
11. [ ] **SecOps**
    1. [ ] **SAST - SonarQube,**
    2. [ ] **SCA - Trivy, Docker Scout**
    3. [ ] **DAST - OWASP ZAP,**
       Configure:
    - [ ] Non-blocking scans
    - [ ] Report storage
12. [ ] **Container registry - Harbor**
13. [ ] **DBMS - Phpmyadmin/Adminer
    1. [ ] Secure Access
14. [ ] Other more utilities tools.

---

11. SRE Observability Stack - Prometheus, Grafana/ELK/Datadog

End-to-end visibility - Centralized monitoring, logging, tracing, and metrics system across hybrid infra. For core SRE Metrics - Latency, Traffic, Errors, Saturations. Detect issues early, Monitor, Improve reliability - SLIs, SLOs, SLAs, Error Budget, MTTR, MTBF, Minimize toils

Stack: (OTEL-LGTM). Metrics → Prometheus, - Visualization → Grafana, Logs → Centralized logging (ELK / Loki)

Telco-Specific Metrics:

- RTP packet loss
- jitter
- bitrate
- session counts

Monitoring and Observability tools - OTEL, Prometheus-LTGM Stack,

- [ ] Deploy Prometheus
  - [ ] Integrate prometheus in the workloads,
  - [ ] Test metrics fetch
- [ ] Deploy Grafana
  - [ ] Setup dashboards:

- [ ] CPU / Memory
- [ ] Network throughput
- [ ] Metrics (Latency, Traffic, request, Errors)
- [ ] SRE Measures and Thresholds (SLIs, SLOs, SLAs, Error Budgets)
- [ ] SRE Actions (MTTR, MTBF, Toil Percent)

- [ ] Deploy logging stack (ELK / Loki)
  - [ ] Setup Traces
  - [ ] Net Hops
  - [ ] Log Retention
- [ ] Add telco specific metrics:
  - [ ] RTP packet loss
  - [ ] jitter
  - [ ] bitrate
  - [ ] session count

---

12. **Cluster Visualization and management (Headlamp and Teleport)**

Tools to visualize and securely manage Kubernetes clusters for Easier debugging, Secure access control, Better visibility. Need controlled access

- [ ] Deploy Headlamp (UI)
- [ ] Configure Teleport:
  - [ ] Secure cluster access
  - [ ] Setup RBAC
  - [ ] Setup Auth
- [ ] Validate:
  - [ ] Access control
  - [ ] Audit logs

---

13. Standardization

- [ ] **Build Standards**
  - [ ] Base container images (distroless)
  - [ ] Security baselines

- [ ] Logging & metrics standards
- [ ] **Deployment Standards**
  - [ ] Helm charts / templates
  - [ ] Environment configurations
  - [ ] Naming conventions
- [ ] **Network/Traffic Optimization**
  - [ ] Host networking
  - [ ] Reduced NAT layers
  - [ ] Reduce cross-region traffic
  - [ ] Optimize ingress/egress paths
- [ ] Strengthen:
  - [ ] VPN / Direct Connect
- [ ] Ensure:
  - [ ] Low-latency hybrid communication

Platformization if required (Future)

Internal Developer Platform (IDPs),

- [ ] Introduce:
  - [ ] Service templates
  - [ ] Prebuilt pipelines
  - [ ] Deployment abstractions
    Using: using Backstage, Crossplane.

Mental Model / Flow:

```
                    ┌──────────────────────────┐
                    │        Clients           │
                    │  Web / Mobile / SIP      │
                    └──────────────────────────┘
                               │
                               │
                     (HTTPS / WSS / SIP)
                               │
                               ▼
              ┌──────────────────────────────┐
              │        AWS CLOUD             │
              │------------------------------|
```

```
                  │                              │
                  │  Ingress / Load Balancer     │
                  │          (ALB)               │
                  │              │               │
                  │      ┌───────▼────────┐      │
                  │      │ Signaling/API/ │      │
                  │      │Web applications│      │
                  │      │   (K8s)        │      │
                  │      └────────────────┘      │
                  │              │               │
                  │              │               │
                  │     (Control / Events)       │
                  └──────────────┼───────────────┘
                                 │
                                 │
                     (VPN / WireGuard Tunnel)
                                 │
┌────────────────────────────────▼──────────────────────────────┐
│                  ON-PREM DATACENTER                           │
│──────────────────────────────────────────────────────────────│
│                                                               │
│    ┌──────────────────────────────────────────────┐           │
│    │        Media Cluster (K8s - Host Net)        │           │
│    │──────────────────────────────────────────────│           │
│    │ RTP / SFU / TURN Servers                     │           │
│    │ High UDP Traffic Handling                    │           │
│    └──────────────────────────────────────────────┘           │
│                   │                                           │
│                   │                                           │
│          ┌────────▼──────────┐                                │
│          │  Hub Cluster      │                                │
│          │───────────────────│                                │
│          │ ArgoCD (GitOps)   │                                │
│          │ Harbor (Registry) │                                │
│          │ MinIO (Storage)   │                                │
│          │ Observability     │                                │
│          └───────────────────┘                                │
│                   │                                           │
│            (Internal Platform)                                │
└───────────────────────────────────────────────────────────────┘
```

Flow Explanation (Simple)

1. Client → AWS (Signaling/API)
2. AWS → decides session/media routing
3. Media traffic → On-Prem (RTP cluster)
4. Control remains in AWS
5. GitOps controls both environments centrally.